

EvalHub Blogs Red Hat Summit 2026

EvalHub Blogs for Red Hat Summit 2026

Blogs

Blog 1 – "From CR to Evaluation Run: How the EvalHub Kubernetes Controller Works" A technical deep dive for platform engineers and developers.

Blog 2 – "EvalHub: Because 'Looks Good to Me' Isn't a Benchmark" A market-positioning piece for technical leads and product teams. Structured around the five interconnected problems in enterprise AI evaluation (fragmentation, domain specificity, reproducibility, documentation/governance, dev-to-enterprise gap) and how EvalHub's specific mechanisms address each – Collections, risk category automation, MLflow-backed reproducibility, and BYOF.

Blog 3 – "EvalHub: Stop Guessing. Start Measuring" An editorial piece on methodology with EvalHub as the practical implementation. Covers the EDD three-step cycle (define → measure → iterate), the sliced evaluation pattern with concrete tables (electronics/clothing/home goods, per-specialty healthcare), and how EvalHub support each step – Collections for criteria definition, the 6-provider ecosystem for multi-dimensional measurement, MLflow for experiment history, and the laptop-to-cluster SDK continuity.

Blog 4 – "EvalHub: Define What "Good" Looks Like" A practitioner's guide to Evaluation Collections for senior engineers and platform architects selecting or validating models for production. Uses the Leaderboard v2 system collection as a concrete, grounded example to explain EvalHub's two-tier threshold system: per-benchmark gates that surface where a model needs work, and a collection-level pass criterion that answers the deployment question. Walks through creating a custom collection in YAML, calibrating weights for a specific use case, running it against a model endpoint via CLI and REST, and interpreting the resulting two-tier score.

Blog 5 – "EvalHub: Bring Your Own Framework – Plug Any Evaluation Tool Into the Platform" A developer-focused how-to for platform engineers and ML practitioners whose evaluation framework is not in EvalHub's default provider set. Introduces the BYOF pattern: implement one abstract method, package it as a container image, and EvalHub treats your framework as a first-class provider. Walks through the three core interfaces – JobSpec (what the platform passes in), JobCallbacks (how to report status and

persist artifacts), and JobResults (what to return) – with a complete, copy-paste-ready adapter implementation.

Blog 6 – "EvalHub from the CLI: Integrate AI Evaluations Into Your CI/CD Pipeline" A practical how-to for DevOps and platform teams to integrate AI model evaluations into CI/CD using the evalhub CLI. The focus is on the CLI as the pipeline integration surface, covering installation, configuration, secret management, and the full evaluation lifecycle (`eval run`, `status`, `results`). It centers on the `--wait` flag as the simple, non-scripted pipeline gate, exiting non-zero on failure. Includes copy-paste examples for GitHub Actions and GitLab CI, a reusable shell script for custom control, and notes on token management, async/blocking patterns, and the MLflow experiment store.

Blog 7 – "EvalHub: Evaluation Results as Immutable OCI Artifacts" This technical post targets platform engineers and compliance teams needing durable, tamper-evident evaluation evidence, moving beyond mutable MLflow records for regulated workloads (EU AI Act, FedRAMP, SOC 2). It explains EvalHub's OCI persistence layer. The post details artifact retrieval via `oras pull` and digest checks, showing how the `oci_ref` links deployed models to immutable evidence.

Demo videos

- Demo Video (draft3) [RHAI Evaluations with EvalHub \(draft3\).mp4](#)
 - Demo deck:  WC - RHAI Summit 2026 (EvalHub demo)

Pending blog topics (post Red Hat Summit due to time constraints):

- EvalHub Agent Evaluation with CLEAR
 - Potential co-authors: members of the ACE team & IBM research, Matteo, Rui, Marius
- Evaluating Embedding Models with EvalHub
 - Potential co-authors:
- Risk assessment and Red Teaming with EvalHub
 - Potential co-authors: Rui, Alessandro, Stuart

Blog01: From CR to Evaluation Run

From CR to Evaluation Run: How the EvalHub Kubernetes Controller Works

By [William Caban](#) [Marius Danciu](#) [Rui Vieira](#) [Julian Payne](#) [Matteo Mortari](#)

A deep dive into the Kubernetes-native architecture powering EvalHub — Red Hat's unified AI evaluation platform.

Why Kubernetes-Native Matters for AI Evaluation

Running AI evaluations in production is not a one-shot script. It is a continuous operational discipline: scheduling compute-intensive jobs, managing concurrency, tracking experiments across model versions, enforcing resource quotas of workloads competing for precious Accelerator resources, and surviving cluster restarts without losing state. If you have operated production ML workloads, you already know that gluing together cron jobs, shell scripts, and a shared Jupyter notebook is not a strategy — it is technical debt on a countdown timer.

EvalHub is built cloud-native and for Kubernetes from the ground up. That means evaluations are not run *on* Kubernetes as an afterthought — the cluster is the control plane. This post walks through exactly how that works, from the Custom Resource Definition (CRD) down to the executor that runs your benchmarks.

The Big Picture: A Two-Layer Control Plane

EvalHub's Kubernetes architecture is organized around two distinct layers.

The first layer is the **TrustyAI Service Operator**, a standard Kubernetes operator that manages the lifecycle of EvalHub instances themselves — deploying the service, configuring environment variables (including the MLflow tracking URI), controlling replica counts, and ensuring the deployment stays converged against the desired state declared in a `EvalHub` Custom Resource.

The second layer is the **EvalHub Server**, which acts as the evaluation orchestration control plane once running. It exposes a versioned REST API (`/api/v1/...`), receives evaluation requests from

any client (code, notebook, CI pipeline, or UI), and fans them out to the appropriate evaluation backend via its Executor Factory.

These two layers interact through the standard Kubernetes declarative model: you declare what you want, the operator ensures it exists, and the server handles the runtime logic.

The EvalHub Custom Resource

Deploying EvalHub onto a Kubernetes or OpenShift cluster is a single YAML declaration, managed by the TrustyAI Service Operator:

```
None
apiVersion: trustyai.opendatahub.io/v1alpha1
kind: EvalHub
metadata:
  name: evalhub
  namespace: my-namespace
spec:
  replicas: 1
  env:
    - name: MLFLOW_TRACKING_URI
      value: "http://mlflow:5000"
```

The operator watches for resources of this kind and reconciles them into a running deployment. The `EvalHub` CRD is part of the `trustyai.opendatahub.io/v1alpha1` API group — the same group that governs TrustyAI's fairness, explainability, and guardrails services. This co-location is intentional: EvalHub is a peer component in the TrustyAI responsible-AI ecosystem, not a standalone sidecar.

Inside the EvalHub Server

Once the operator has brought the `EvalHub` pod online, the server itself handles the runtime orchestration. The server is written in Go and uses the standard `net/http` router, structured JSON logging via `zap`, and Prometheus metrics — making it operationally compatible with existing OpenShift observability stacks without configuration overhead.

The storage layer is abstracted behind a pluggable interface: SQLite in-memory for local development and single-node testing, and PostgreSQL for production multi-replica deployments. Configuration is loaded from `config/config.yaml`, and credentials are provided in Kubernetes secrets.

Providers — the evaluation backends — are defined in YAML configuration files shipped with the TrustyAI operator. The default set includes `lm-evaluation-harness` (with 167 benchmarks), `Garak`, `GuideLLM`, and `LightEval`. Additional providers, such as `RAGAS`, `MTEB` and `IBM CLEAR`, can be integrated via the EvalHub Contrib adapters, and Providers can be registered initially by adding a YAML entry to the `providers` ConfigMap, or later using the API — no code changes required. The new provider immediately appears through the standard `/api/v1/evaluations/providers` and `/api/v1/evaluations/benchmarks` endpoints.

The Executor Factory: How Evaluation Backends Are Invoked

As evaluation job requests arrive at the EvalHub service, we first identify the backend runtime, an internal abstraction designed to accommodate the flow based on the environment in which the service runs. Thus, as EvalHub is deployed via the ODH operator, the runtime environment is Kubernetes (OpenShift cluster). For local execution (e.g. laptop/workstation/server), we have a different runtime implementation that does not rely on Kubernetes at all. Assuming a Kubernetes context, each evaluation/benchmark is scheduled as a separate Kube Job and runs in isolation from other benchmarks as part of the overall evaluation or collection. On-the-fly progress and status information is fed back into the service by a sidecar container that insulates the evaluation container from many details, making it easier for adopters to write their own custom evaluations and integrate them into EvalHub.

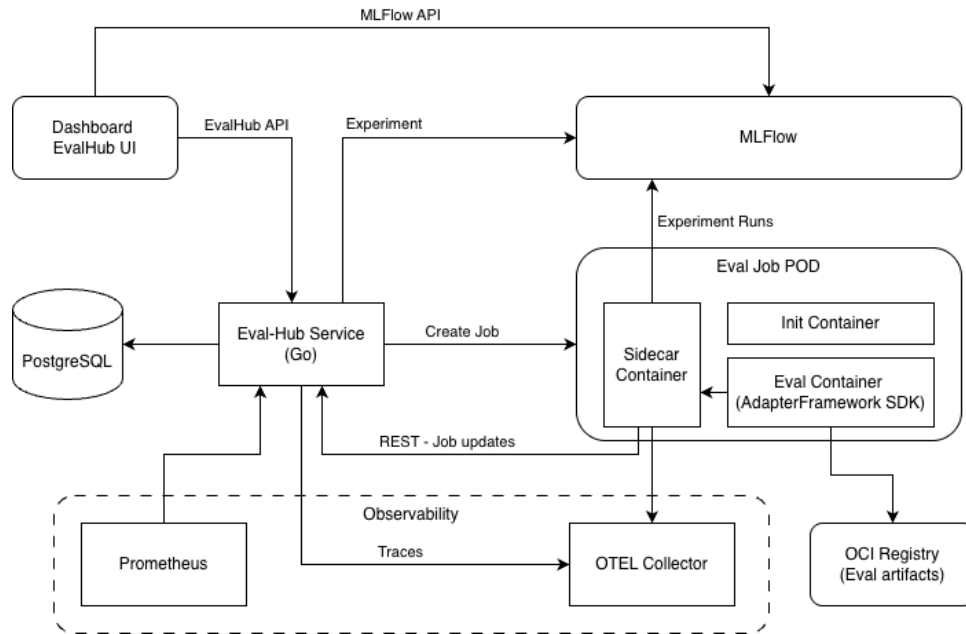


Fig 1 - EvalHub architecture

EvalHub supports four categories of executors today:

1. Im-evaluation-harness — the default for capability benchmarks. The server routes requests to EleutherAI's Im-evaluation-harness library, which runs tasks in batch and returns structured results. Collection-based evaluations group all Im-eval tasks into a single backend call for efficiency, avoiding the overhead of separate invocations per benchmark.

2. LightEval — for fast, lightweight capability evaluations. LightEval is registered as a first-class provider in EvalHub's default provider set, and its execution model is the **BYOF adapter pattern applied at platform scale**. The server provisions a Kubernetes Job running the pre-built `quay.io/evalhub/community-lighteval:latest` container image with `python main.py` as its entrypoint. That container reads its `JobSpec` from a ConfigMap mounted at `/meta/job.json`, runs the LightEval evaluation framework in-process, and reports status and results back to EvalHub via the sidecar callback URL. No KubeFlow Pipelines endpoint is involved. The provider YAML declares this deployment model:

None

```

id: lighteval
runtime:
  k8s:

```



```
image: quay.io/evalhub/community-lighteval:latest
entrypoint: [python, main.py]
cpu_request: 100m
memory_request: 128Mi
```

This is the same pattern any BYOF adapter uses — the difference is that LightEval ships as a ready-made adapter image rather than requiring teams to build their own. If you want to build a custom LightEval integration via the SDK's `FrameworkAdapter`, the execution is identical: `run_benchmark_job` calls LightEval's Python library in-process inside your adapter pod.

3. GuideLLM — for infrastructure performance profiling. GuideLLM measures throughput, latency, memory usage, and token cost across hardware configurations, making it the right tool when the question is not "how accurate is the model?" but "how fast, at what cost, and on which hardware?" Like other providers, GuideLLM is declared in a YAML configuration file and exposed through the standard `/api/v1/evaluations/providers` endpoint.

4. Custom / Bring Your Own Framework (BYOF) — via the EvalHub SDK's `FrameworkAdapter` base class. You implement a single `run_benchmark_job` method, use the provided callbacks for progress reporting, and register the provider via `ConfigMap`. EvalHub handles scheduling, status tracking, and result aggregation automatically.

The EvalHub SDK: Adapter, Client, CLI, and MCP Server

The `eval-hub-sdk` (github.com/eval-hub/eval-hub-sdk) is a Python package (`pip install eval-hub-sdk`) with a registered `evalhub` CLI entry point and four modules under `src/evalhub/`:

`evalhub.adapter` — the BYOF framework adapter layer. Implement `run_benchmark_job()`, use `JobCallbacks` for status reporting and OCI artifact persistence, and your evaluation logic runs as a portable adapter. In production, EvalHub deploys it as a Kubernetes job pod and manages all scheduling and result aggregation; locally, the adapter can be instantiated directly for development and testing. The adapter also includes built-in MLflow integration (`callbacks.mlflow.save(...)`) supporting both the ODH client and the upstream `mlflow` library.

evalhub.client — typed sync and async REST clients (`EvalHubClient`, `AsyncEvalHubClient`) for submitting jobs, listing and managing providers, benchmarks, and collections programmatically. Supports multi-tenant deployments via a per-request `tenant` parameter.

evalhub.cli — a fully implemented CLI (`evalhub`) built on Click, with command groups and direct commands:

- `evalhub eval run/status/results/cancel` — submit jobs (from YAML/JSON config or inline flags), watch status with `--wait/--watch`, retrieve results in `table/JSON/YAML/CSV`
- `evalhub collections list/describe/create/delete/run` — manage and execute benchmark collections
- `evalhub providers list/describe` — inspect registered providers and their benchmarks
- `evalhub health` — check service availability with response time
- `evalhub config set/get/list/use` — manage multi-profile config at `~/.config/evalhub/config.yaml`
- `evalhub mcp` — start the MCP server in stdio transport mode

evalhub.mcp (dev preview) — an MCP server built on `FastMCP`. Exposes 9 MCP resources (providers, benchmarks, collections, jobs — all browsable by AI agents) and 2 MCP tools (`submit_evaluation`, `cancel_job`). Resource template parameters include autocomplete support. Started via `evalhub mcp` or used directly as an `AsyncEvalHubClient` wrapper.

OCI Artifact Persistence

The adapter's OCI interface is fully implemented — not a placeholder. `OCIArtifactPersister` uses `olot` to create a compliant OCI layout on disk, then `oras-py` to push it to a real registry. In Kubernetes mode, the sidecar acts as an OCI proxy so the adapter doesn't need to handle registry credentials directly — authentication is delegated to the sidecar. In local mode, standard Docker config file authentication is used.

None

```
artifact = callbacks.create_oci_artifact(OCIArtifactSpec(
```

```

files_path=Path("./results/"),
coordinates=OCICoordinates(
    oci_host="quay.io",
    oci_repository="myorg/eval-results",
    annotations={"score": "0.85"}
)
))
# artifact.reference → "quay.io/myorg/eval-results:evalhub-<job_sha>@sha256:..."
# artifact.digest → "sha256:..."

```

The OCI artifact reference — host, repository, and SHA256-based tag derived from job ID, benchmark ID, provider ID, and benchmark index — is then included in the `report_results()` call back to EvalHub, making it queryable alongside MLflow experiment data.

MLflow: The Experiment Memory

When an evaluation request includes an experiment configuration, the EvalHub server creates or reuses an MLflow experiment and associates the run with it. The caller provides the experiment name and optional tags and the server injects additional metadata automatically, so that every tracked run is attributable without manual bookkeeping.

```

JSON
{
  "name": "llama-3-healthcare-safety-v2",
  "tags": [
    {"key": "environment", "value": "production"},
    {"key": "model_family", "value": "llama"},
    {"key": "collection", "value": "healthcare_safety_v1"}
  ]
}

```

If the named experiment already exists and is active, EvalHub reuses it; otherwise it creates a new one. In multi-tenant deployments, experiments are scoped to the tenant's namespace so that job pods can reach the tracking server with matching credentials.

The adapter SDK's built-in MLflow integration logs the evaluation metrics, job configuration, and model information to the experiment run. Adapters can also persist additional metadata and artifacts, such as per-sample predictions, confusion matrices, or custom analysis outputs. This gives teams full control over what is captured alongside the standard evaluation results.

Platform teams have a single queryable store for evaluation history across model versions, benchmark suites, and time. The experiment record captures the exact configuration that produced a given set of scores, making reproducibility a first-class outcome rather than an afterthought.

Observability Out of the Box

EvalHub ships with three observability surfaces:

Health checks — Kubernetes liveness and readiness probes are configured in the deployment manifests, ensuring the operator restarts unhealthy pods and the load balancer only routes traffic to ready instances.

Prometheus metrics — the server instruments request counts, request duration, and evaluation statistics at the `/metrics` endpoint in standard Prometheus and OTEL exposition format. These can be scraped by OpenShift Monitoring without any additional configuration.

Structured logging — all logs are emitted as structured JSON in production mode (or human-readable console output in development), with request IDs and evaluation IDs in every log line for correlation across distributed traces.

Horizontal Scaling

Because EvalHub's state lives in PostgreSQL (in production) and all experiment data in MLflow, the server itself is stateless and horizontally scalable. The EvalHub CR `spec.replicas` field controls the pod count directly. Combined with Kueue's `ClusterQueue` capacity enforcement, scaling the evaluation control plane is a matter of adjusting a single integer and a resource quota — no distributed coordination code required.

Multi-Tenant Architecture

EvalHub is designed for shared-platform deployments in which multiple teams, projects, or business units require isolated evaluation environments within the same cluster. The multi-tenancy model is implemented at two levels that work together.

API-Level Tenant Isolation

At the REST API layer, every request includes an `X-Tenant` HTTP header that identifies the requesting tenant. The storage layer exposes a chainable builder interface — `WithTenant()` — so every SQL query is automatically filtered by tenant scope without requiring per-handler logic. Providers can be registered as `system`-scoped (shared, read-only across all tenants) or `tenant`-scoped (private, mutable by the owning tenant). MLflow experiment namespacing follows the same boundary: each evaluation run is scoped to a tenant-specific namespace so that job pods running in that namespace can reach the tracking server with matching credentials. The SDK's `EvalHubClient` and `AsyncEvalHubClient` accept a `tenant` parameter at initialization — set once, propagated as `X-Tenant` on every request — with a per-call override for cases where a single client needs to act across tenants.

Kubernetes Namespace-Level Isolation

On the infrastructure side, the EvalHub operator uses a label-driven discovery model rather than an explicit tenant registry. Any Kubernetes namespace labeled `evalhub.trustyai.opendatahub.io/tenant=true` is automatically onboarded as a tenant. The operator's `reconcileTenantNamespaces()` function watches `Namespace` resources, filters by that label, and provisions the required objects in each tenant namespace: a dedicated job `ServiceAccount`, `RoleBindings` granting the API `ServiceAccount` permission to create and manage `Jobs` and `ConfigMaps` in that namespace, and a `Service CA ConfigMap` for TLS callback injection on OpenShift. This means adding a new tenant requires no changes to the EvalHub deployment itself — administrators or operators label a namespace and the controller converges automatically. Resource quota enforcement at the namespace level relies on standard Kubernetes `ResourceQuota` objects, which can be applied independently per tenant namespace by the platform team without EvalHub-specific configuration.

Together, the two layers give platform teams a clean operational model: the `X-Tenant` header enforces data isolation at the application layer, and the namespace label mechanism enforces compute and credential isolation at the infrastructure layer.

The EvalHub tenancy model is aligned with the MLFlow tenancy model when MLFlow is deployed with *Kubernetes-Workspace-Provider*. Before the request reaches the EvalHub API layer, we have an authentication and authorization RBAC layer that performs token review and subject access review. Fundamentally, there are 3 types of kube resources protected by RBAC: evaluations,

providers, and collections. Thus, an admin can create custom Roles to grant fine-grained access to specific identities such as service accounts, users, or groups.

Putting It All Together

The path from a CR to a completed evaluation run looks like this:

1. A user or CI system submits a POST to `/api/v1/evaluations` with a model endpoint, some benchmarks, or a collection ID, and an experiment name.
2. The EvalHub server validates the request, expands the collection into individual benchmarks, groups them by provider, and creates an async evaluation job with a unique ID.
3. The Executor Factory dispatches each provider group to the appropriate backend adapter — lm-eval tasks go to the lm-evaluation-harness runtime, performance profiling to GuideLLM, and custom evaluations to any registered BYOF adapter.
4. Results flow back and are aggregated using the collection-level weights declared in the collection YAML, producing a final composite score.
5. The MLflow client writes the full experiment record — scores, parameters, hardware context — to the tracking server.
6. The caller polls `/api/v1/evaluations/{id}` for status, or queries MLflow directly for historical comparison.

At every step, standard Kubernetes primitives — CRDs, operators, resource quotas, Prometheus, structured logs — do the heavy lifting. EvalHub does not reinvent cluster operations. It brings evaluation into the operational model you already have.

The Kubernetes controller, the `EvalHub` CRD, and the full SDK — CLI, MCP server (dev preview), adapter with OCI artifact persistence, and REST client — all represent the current state of the platform.

If you want to explore the code, start with the [main eval-hub documentation site](#), and continue with [eval-hub](#) server repo, the [eval-hub-sdk](#) for the Python REST client, the BYOF adapter, the CLI, the MCP server, the OCI artifact interface, and the `trustyai-service-operator` for the controller implementation.

Blog02: Because "Looks Good to Me"

EvalHub: Because "Looks Good to Me" Isn't a Benchmark

By [William Caban](#) [Rui Vieira](#)

Why AI evaluation is broken in most enterprises — and what a unified platform actually fixes.

The Confidence Gap

Your team just finished three months building a RAG-powered assistant for customer support. It handled every test query you threw at it in the demo. Leadership is excited. The launch date is set.

Then it goes live. Users complain that answers about product returns are confidently wrong. The legal team flags a response citing a policy that was deprecated 18 months ago. The model is fast, articulate, and unambiguously unreliable.

What went wrong? Most likely, nothing catastrophic and nothing subtle. The evaluation process simply never asked the right questions. "Looks good to me" and a handful of manual spot checks stood in for systematic measurement. The team optimized for the demo, not for the deployment.

This is not a story about one team's bad luck. It is the dominant pattern in enterprise AI development today, with five distinct structural causes.

Five Problems That Break AI Evaluation at Scale

1. The Tooling Fragmentation Problem

A typical enterprise AI team evaluating a new model or RAG pipeline might use EleutherAI's LM Evaluation Harness for capability benchmarks, RAGAS for retrieval quality, Garak for red-teaming and safety probes, and GuideLLM for throughput and latency profiling — each tool run separately, by different team members, with results stored in different places. Stitching these together into a coherent picture requires custom

scripts and manual data wrangling, and the burden of maintaining this evaluation infrastructure falls on someone whose primary role is performing evaluations, diverting their focus.

The operational cost is real: evaluation runs are skipped under deadline pressure, results are not compared across model versions because the formats are incompatible, and the institutional knowledge of "what we ran last time" lives in one engineer's notebook.

2. The "What Should I Measure?" Problem

Generic evaluation tools are built around academic benchmarks: MMLU, ARC, HellaSwag, HumanEval. These are valuable, but they answer the wrong question for most enterprise use cases. A healthcare LLM needs to be evaluated on clinical reasoning accuracy and regulatory compliance, not on general trivia. A multilingual customer service agent needs a per-language breakdown, not a single aggregate accuracy score.

The result is what might be called the vanity metric trap: teams report high scores on benchmarks that do not predict real-world performance in their specific domain, and ship systems that fail in the exact ways the benchmark did not measure.

3. The Reproducibility Crisis

AI model behavior is sensitive to hardware, driver versions, quantization settings, batch sizes, and prompt templates. A benchmark score produced on an A100 cluster may not reproduce on a T4 node. A score from last quarter may not reproduce today if the model's serving configuration has changed.

Without a systematic way to capture and attach environment metadata to evaluation results – hardware specs, software versions, model config – benchmark scores are claims, not evidence. This matters enormously for regulated industries: the EU AI Act and FedRAMP require documented, reproducible evidence of model behavior, not a dashboard screenshot.

4. The Documentation and Accessibility Gap

Evaluation knowledge tends to scatter: MLflow experiments with results no one outside the ML team can query, model cards written once and never updated, and benchmark configurations in READMEs that

silently go stale. The consequence is that decisions about which model to deploy, which component to tune, or which risk to accept are made from tribal knowledge rather than traceable evidence.

This is not a data quality problem. It is a workflow problem: there is no standard, organization-wide format for evaluation reports, no single place to look, and no automated path from a completed evaluation run to a governance-ready artifact.

5. The Dev-to-Enterprise Gap

A developer can run a quick evaluation from a Python script on their laptop in minutes. Moving that same evaluation to a Kubernetes cluster for reproducible, production-scale runs – with proper concurrency management, resource quotas, experiment tracking, and structured results – requires either a platform engineering investment or months of glue code. Most teams never bridge this gap. Their evaluation capability stays at the laptop-script level even as their model deployments scale to production traffic.

The gap means that evaluation remains a manual, developer-local activity that does not fit into CI/CD pipelines, does not run at the scale of production data, and does not produce the governance artifacts the enterprise actually needs.

Introducing EvalHub

EvalHub is the Red Hat AI unified AI evaluation platform. It directly addresses all five problems above through a single orchestration control plane and a set of deliberately designed primitives.

A Single Orchestration Layer for Every Framework

EvalHub's core is a REST API server – written in Go, deployed on Kubernetes via the TrustyAI operator – that routes evaluation requests to any supported backend without the calling code needing to know which framework is actually running. The default provider set, declared in YAML configuration files, includes:

- **lm-evaluation-harness** – 167 benchmarks for capability, reasoning, and knowledge evaluation
- **Garak** – red-teaming, safety probes, and toxicity detection
- **GuideLLM** – throughput, latency, and infrastructure profiling
- **LightEval** – fast, lightweight capability benchmarks

- **MTEB** – massive text embedding benchmarks

A single POST request to `/api/v1/evaluations` can fan out to multiple backends in parallel, aggregate the results with configurable weights, and write the full experiment record to MLflow – all without the caller managing any of that complexity.

Adding a new framework does not require a code change: you extend the `FrameworkAdapter` base class from the EvalHub SDK, implement a single `run_benchmark_job` method, and register the provider via a `ConfigMap` entry. EvalHub handles scheduling, status reporting, and result aggregation from that point forward.

The SDK (github.com/eval-hub/eval-hub-sdk) packages four capabilities alongside the adapter:

- **evalhub.client** – sync and async typed REST clients for submitting jobs and querying providers, benchmarks, and collections from Python code or notebooks
- **evalhub.cli** – a fully featured `evalhub` CLI with commands for running evaluations, watching status, retrieving results, managing collections and providers, and checking service health; supports YAML/JSON config files, multi-profile configuration, and `--wait/--watch` flags for blocking runs
- **evalhub.mcp** (dev preview) – a FastMCP-based MCP server exposing 9 browsable resources and 2 action tools, started via `evalhub mcp`; lets AI agents and coding assistants invoke evaluations and retrieve results via the Model Context Protocol
- **OCI artifact persistence** (via `evalhub.adapter`) – evaluation results are pushed to an OCI registry using `olot` and `oras`, with SHA256-derived tags and full annotation support; in Kubernetes mode, the sidecar handles registry authentication transparently

Evaluation Collections: Answering "What Should I Measure?"

The platform's answer to the domain specificity problem is **Evaluation Collections** – named, versioned, expert-curated sets of benchmarks that are framework-agnostic and tailored to specific verticals and use cases.

A collection is declared with a list of benchmarks, their providers, and their relative weights. A `healthcare_safety_v1` collection, for example, might include clinical reasoning benchmarks from

lm-evaluation-harness, safety probes from Garak, and RAG-groundedness metrics from RAGAS – each weighted according to the use case's risk profile.

Calling a collection is as simple as including its ID in an evaluation request:

```
Shell
curl -X POST /api/v1/evaluations \
-d '{
  "name": "llama-3-v2-healthcare-eval",
  "model": { "url": "http://vllm-service:8080/v1" },
  "collection": { "id": "healthcare_safety_v1" },
  "experiment": { "name": "llama-3-v2-healthcare-eval" }
}'
```

EvalHub looks up the collection, extracts all benchmarks, groups them by provider to improve execution, and runs them – returning a weighted aggregate score along with a per-benchmark breakdown. The collection encodes organizational evaluation knowledge in a shareable, versionable, reusable form.

Built-In Reproducibility and Governance

Every evaluation run is automatically tracked in MLflow with a structured `ExperimentConfig` that captures the experiment name, tags (environment, model family, collection version, etc.), and the full configuration used to produce the results. This gives teams a queryable, historical record of every evaluation run – the foundation for regression tracking, model comparison, and governance reporting.

The OCI artifact persistence feature establishes a tamper-evident connection between a model artifact and its deployment justification. This is achieved by directly embedding evaluation results and traces as metadata within the OCI ModelCar images, ensuring that the evaluation evidence is persistently linked to the deployed model.

From Laptop to Cluster Without Friction

The EvalHub SDK provides the same evaluation interface whether you are running a quick check in a Python notebook, wiring evaluations into a CI/CD pipeline, or scheduling a production evaluation run on an OpenShift cluster. The server handles the translation from a structured API call to the Kubernetes

primitives that manage execution at scale – including Kueue-based resource quota enforcement, pod scheduling, and status tracking via Custom Resources.

A developer does not need to understand Kubernetes to run an evaluation. A platform team does not need to build custom evaluation infrastructure. The same EvalHub instance serves both.

What EvalHub Is Not

EvalHub is not a benchmark leaderboard. It is not a replacement for framework-specific tools like lm-evaluation-harness or RAGAS. Those tools do the actual measurement work; EvalHub orchestrates them, routes between them, tracks their outputs, and makes their results governable.

EvalHub is rapidly evolving to keep pace with the swift developments in the AI world. It offers a sophisticated solution that addresses the five previously mentioned challenges more comprehensively than any single evaluation framework. EvalHub delivers a robust server for routing to live backends, tracking genuine experiments, and deploying on actual Kubernetes clusters.

Getting Started

EvalHub is open source under the Apache 2.0 license. The server lives at github.com/eval-hub/eval-hub. The Python SDK – evalhub CLI, REST client, BYOF adapter, MCP server, and OCI artifact persistence – is at github.com/eval-hub/eval-hub-sdk. The TrustyAI operator that manages its Kubernetes deployment is at github.com/trustyai-explainability/trustyai-service-operator.

For teams already running Open Data Hub or Red Hat OpenShift AI, EvalHub deploys as a component of the TrustyAI stack – no separate infrastructure required.

If your current evaluation process involves a combination of "it looked good in testing," a few MLflow runs that nobody queries, and a model card written three months ago, EvalHub exists specifically to replace that with something you can actually trust.

Blog03: EvalHub: Stop Guessing. Start Measuring.

EvalHub: Stop Guessing. Start Measuring.

By **William Caban** **Matteo Mortari**

How Evaluation-Driven Development and sliced evaluation turn AI optimization from an art into an engineering discipline – and how EvalHub makes it practical.

The Problem with Building AI Like It's Software

If you have shipped software, you know TDD. Write a failing test. Write the code to make it pass. Refactor. Ship with confidence. The red-green-refactor cycle is elegant because it is prominently deterministic in nature: the test either passes or it doesn't. Every state is unambiguous.

AI systems do not work that way.

A language model answering a customer support question might give ten different responses to the same query across ten runs – all of them arguably correct, none of them identical. "Does it work?" is the wrong question. The right question is "how often does it work, for whom, in what contexts, and how do we know?" Programmatic pass/fail can't answer that. Threshold-based, multi-dimensional, gradient scoring can.

This is the insight behind **Evaluation-Driven Development (EDD)**: an evolution of TDD for probabilistic systems that replaces plain assertions now with scored, multi-dimensional evaluation criteria, and replaces green/red builds with measurable gaps between current performance and desired thresholds.

The EDD Cycle

EDD has three steps, each with an analog in traditional software engineering, and each qualitatively different for AI.

Step 1: Define Evaluation Criteria – Before You Write a Prompt

In TDD, you write tests before you write code. In EDD, you define evaluation criteria before you write prompts, choose a model, or design a pipeline.

This sounds obvious. It almost never happens. The default process in most AI projects is: build something, prompt it until it seems to work, demo it, ship it, and then figure out what went wrong in production.

EDD forces the question upfront: what does good look like? Not in the abstract, but specifically and measurably. For a customer support agent, that might mean:

- Factual accuracy: 90% of claims match source documentation
- Escalation rate: fewer than 20% of queries transferred to a human agent
- Response latency: under 2 seconds at the 95th percentile
- Tone: no instances of defensive or dismissive language (measured by a secondary classifier)

These criteria become your objectives. They determine what you measure, how you weight results, and what "done" means for every subsequent iteration.

Step 2: Measure Quality Scores – 0 to 100, Not Pass/Fail

Once the criteria are defined, you run your system and score it. Instead of a simple Pass/Fail, you can define Gradient Scores – percentages, weighted composites, dimensional breakdowns.

A score of 82% on factual accuracy against a 90% threshold tells you something a simple "FAILED" cannot: you are 8 points away, not infinitely far. It tells your engineering team where to focus. It gives you a baseline against which every subsequent change can be compared.

This is the reframe that makes AI development pragmatically and scientifically manageable: instead of a black box that you poke until it seems better, you now have an optimization problem with a measurable current state and a defined target state. Closing the gap is engineering a solution, not (only?) intuition.

Step 3: Iterate on Prompts and Configurations – Based on Data

With scores in hand, you run experiments. You A/B test prompt variations. You swap retrieval strategies. You try different models. You compare results not against your gut feeling but against your baseline scores.

Each iteration either closes the gap, moves you progressively toward your final objective ...or it doesn't. The evaluation tells you which. This transforms prompt engineering from an art into a systematic discipline: controlled experiments with measurable outcomes.

The Sliced Evaluation: Where the Real Work Happens

Here is where EDD reveals its most powerful capability but also where most teams miss the opportunity entirely.

An aggregate accuracy score is a summary: It provides a global aggregate, but might hide interesting and important insights.

For example: consider an e-commerce recommendation system with an aggregate accuracy of 79%. That sounds reasonable. But what if the per-category breakdown looks like this?

Category	Threshold	Score	Status
Electronics	90%	82%	Below target
Clothing	75%	95%	Exceeding
Home Goods	80%	45%	Critical

The aggregate score of 79% is masking a critical failure in Home Goods that affects a third of the catalog. An engineering team looking only at the top-line number would have no idea where to focus, or might even deploy in the production system, confident it is "almost 80%."

Sliced evaluation breaks aggregate scores into the constituent dimensions that actually matter for the specific use case: by product category, by query language, by user segment, by medical specialty, by regulatory domain, etc. Each slice gets its own threshold, its own score, and its own trend over time.

This is the scientific precision that the EDD cycle promises at step 3. You are not optimizing "the system." You are closing the specific gap in Home Goods recommendations, which might mean a different retrieval strategy for product catalog data, additional fine-tuning on product descriptions, or simply a different prompt template for that category!

The same principle applies across domains. A healthcare system might track per-specialty evaluation scores; for example: cardiology at 88% against a 95% threshold, neurology at 97%. A multilingual system tracks per-language accuracy; for example: English at 95%, Mandarin at 89%, and Spanish at 78%. Each slice is an independent optimization target.

EvalHub: EDD as an Engineering Practice

The EDD cycle is a methodology. EvalHub is the platform that helps you turn it into an engineering practice!.

Defining Evaluation Criteria with Collections

EvalHub operationalizes Step 1 through **Evaluation Collections**: named, versioned sets of benchmarks with explicit weighting. A collection is the machine-readable form of your evaluation criteria.

For a healthcare LLM deployment, the collection might look like:

JSON

```
{
  "id": "healthcare_safety_v1",
  "benchmarks": [
    { "id": "medqa", "provider_id": "lm_evaluation_harness", "weight": 2.0, "pass_criteria": {"threshold": 80.0} },
    { "id": "pubmedqa", "provider_id": "lm_evaluation_harness", "weight": 1.5, "pass_criteria": {"threshold": 75.0} },
    { "id": "toxicity", "provider_id": "garak", "weight": 2.5, "pass_criteria": {"threshold": 70.0} },
    { "id": "faithfulness", "provider_id": "ragas", "weight": 1.5, "pass_criteria": {"threshold": 80.0} }
  ]
}
```

The collection is defined before a single evaluation is run. It encodes the team's measurement strategy: which dimensions matter, how they are weighted, ... all in a shareable, versionable artifact. When the criteria change (because the use case evolves, or a new regulatory requirement emerges), the collection can be updated and versioned, not rewritten in someone's script.

Measuring at Scale with Automatic Experiment Tracking

Step 2 “measuring quality scores” is where EDD's operational overhead typically kills the practice. Running five frameworks against three model variants, capturing all configurations, storing results in a queryable format, and making them accessible to the wider team is a significant engineering investment.

EvalHub handles this automatically. A single POST to `/api/v1/evaluations` with a collection ID and a model endpoint:

1. Expands the collection into individual benchmarks grouped by provider
2. Routes each group to the appropriate backend (lm-eval, RAGAS, Garak, etc.) in parallel

3. Applies the collection's weights to produce a dimensional score breakdown
4. Writes the full experiment record, including scores, configurations, and tags to MLflow

The MLflow integration means every evaluation run is automatically tracked with the configuration that produced it: model version, collection version, hardware tags, and environment. Reproducing a result from three months ago is a query, not a detective investigation.

Every provider in EvalHub's default set covers a different evaluation dimension:

- **lm-evaluation-harness (167 benchmarks):** reasoning, knowledge, coding, commonsense
- **RAGAS:** retrieval quality, answer relevance, and faithfulness for RAG pipelines
- **Garak:** red-teaming, toxicity, bias, safety adversarial probes
- **GuideLLM:** latency, throughput, memory usage, hardware efficiency
- **LightEval:** fast capability checks
- **MTEB:** text embedding quality across retrieval tasks

The sliced evaluation that EDD requires (per-category, per-language, per-specialty, etc.) is achieved by running collections against different filtered dataset subsets and tagging the results accordingly in MLflow. Each slice becomes a trackable dimension in the experiment history.

Iterating from the Notebook to the Cluster

Step 3 "iterate based on data" is where EvalHub's architecture eliminates the dev-to-enterprise gap.

A developer running a quick prompt-variation test uses `evalhub.client` (the SDK's typed Python REST client) to submit an evaluation request against an EvalHub instance. The exact same call, pointed at a production EvalHub deployment on OpenShift, runs as a Kubernetes-native job with Kueue-managed scheduling, resource quotas, structured logging, and Prometheus metrics. There is no "evaluation mode for development" and "evaluation mode for production": there is one API, one format, one results store.

This means the EDD iterate-and-measure cycle can happen in CI/CD. The SDK's `evalhub.cli` makes EvalHub *also* directly accessible from shell scripts and pipeline steps without writing Python code; for example: `evalhub eval run --config eval.yaml --wait` submits a job and blocks until it completes, returning a non-zero exit code on failure. Every pull request that touches a prompt template, a retrieval strategy, or a model configuration can trigger an EvalHub evaluation run as part of the pipeline; in

this way, the PR review will include not just code review but an evaluation report: baseline score, updated score, which slices improved and which regressed.

The `evalhub.mcp` module extends this to agentic workflows: AI agents and coding assistants can browse providers, benchmarks, and collections as MCP resources, submit evaluation jobs, and cancel running jobs via two MCP tools: all through a dedicated MCP server. This enables a pattern in which an agent validates its own changes against a defined collection before committing.

The `Bring-Your-Own-Framework` capability rounds this out: extend `FrameworkAdapter` in `evalhub.adapter`, implement `run_benchmark_job`, and your custom evaluation logic plugs into the same orchestration, tracking, and reporting infrastructure. The adapter's OCI persistence (`callbacks.create_oci_artifact(...)`) pushes evaluation results to an OCI registry using `olot` and `oras-py` – in Kubernetes mode, the sidecar handles authentication; in local mode, standard Docker config is used. The artifact OCI reference and digest are included in the results reported back to EvalHub, so long-term evaluation provenance is queryable alongside MLflow experiment data.

EDD in Practice: From E-Commerce to Healthcare

Let's make this concrete with two examples drawn from real evaluation patterns.

E-Commerce Recommendation Quality

The team defines a collection with three slices: Electronics (threshold: 90%), Clothing (threshold: 75%), Home Goods (threshold: 80%). They run the collection against their baseline RAG pipeline. Electronics: 82%, Clothing: 95%, Home Goods: 45%.

The optimization target is unambiguous: Home Goods at 45% needs urgent attention. The team discovers that the product catalog data for Home Goods is poorly structured and rarely retrieved correctly. They redesigned the retrieval strategy for that category specifically: a change that would never have been identified from the 79% aggregate score.

Healthcare LLM Safety Validation

The team uses the `healthcare_safety_v1` collection, which covers clinical reasoning (MedQA, PubMedQA), safety (Garak toxicity and adversarial probes), and RAG groundedness (RAGAS

faithfulness). Each benchmark has a minimum threshold reflecting regulatory requirements. Per-specialty slicing tracks cardiology, dermatology, and neurology separately.

Before each model update, the collection runs automatically. If any specialty falls below its threshold, the update is blocked. If safety scores decline even when aggregate accuracy improves, the evaluation surfaces the tradeoff explicitly rather than hiding it in the average.

The Shift: From Black Box to Optimization Problem

The traditional approach to AI development treats the model as a black box: put something in, get something out, decide subjectively if it's good enough.

EDD reframes the problem: you have a current performance score, a desired performance threshold, and a measurable gap between them. Your engineering work is to close that gap systematically, through controlled experiments, guided by evaluation data rather than instinct.

EvalHub is the platform that makes this reframe possible, pragmatic, and at-scale. It handles the measurement infrastructure so teams can focus on the optimization work. It enforces the discipline of defining criteria before running experiments. It makes sliced evaluation as easy as aggregate evaluation. And it ensures that the evaluation practice scales from a developer's laptop to a production Kubernetes cluster without a platform engineering project in between.

Stop guess-work!. Define your criteria. Run your collections. Measure the gap. Close it. Repeat.

That is EDD. That is what EvalHub is built for.

Start Here

- **EvalHub website:** <https://eval-hub.github.io>
- **EvalHub Server:** github.com/eval-hub/eval-hub
- **EvalHub SDK** (evalhub CLI, REST client, BYOF adapter, MCP server, OCI persistence):
github.com/eval-hub/eval-hub-sdk
- **OpenAPI Specification:** eval-hub.github.io/eval-hub

- **TrustyAI Operator** (for Kubernetes/OpenShift deployment):
github.com/trustyai-explainability/trustyai-service-operator

Blog04: EvalHub: Define What "Good" Looks Like

EvalHub: Define What "Good" Looks Like — Before You Run a Single Benchmark

By [William Caban](#) [Julian Payne](#) [Marius Danciu](#)

How to build custom Evaluation Collections with meaningful thresholds — and why the Leaderboard v2 collection is the right place to start.

The Benchmark Selection Problem

In [Because "Looks Good to Me"](#) we identified five structural failures in enterprise AI evaluation. Problem two — the *"What Should I Measure?"* problem — is the one that bites teams earliest, and quietly. You have a model. You have an endpoint. You need to know if it's good enough to deploy. So you run MMLU, get a score, and make a judgment call.

The score tells you almost nothing useful.

Not because MMLU is a bad benchmark. It is a rigorous one. The problem is that a single benchmark score is a proxy answer to a different question. What you actually need to know is: *for my specific use case, deployment context, and risk tolerance, does this model clear the bar?* That requires a set of measurements — plural — and a defined bar for each one. Not vague intuition about what a "good" score looks like.

[EvalHub: Stop Guessing. Start Measuring.](#) introduced Evaluation-Driven Development and showed how **Evaluation Collections** operationalize Step 1 of the EDD cycle: defining criteria before running experiments. This post goes one level deeper. It covers how to read an existing system collection, understand its threshold logic, and build your own collection that encodes your actual measurement strategy — with thresholds that mean something.

The Leaderboard v2 collection is the worked example. It is one of EvalHub's built-in system collections, and its threshold choices are defensible enough to deserve explanation.

What an Evaluation Collection Actually Is

An Evaluation Collection is a named, versioned artifact that captures three things:

1. **Which benchmarks to run** — identified by benchmark ID and the provider that executes them
2. **How to weight them** — numerical weights that determine each benchmark's contribution to the aggregate score
3. **What thresholds define a pass** — both per-benchmark (individual gates) and collection-level (overall pass criteria)

Collections are stored server-side and referenced by ID in evaluation requests. They are not shell scripts or experiment configs — they are first-class EvalHub resources with full CRUD support via the REST API and CLI.

The server distinguishes two scopes:

- **system** — built-in collections shipped with EvalHub and maintained by the project team. Read-only. Examples: `leaderboard-v2`.
- **user** — collections you create. Full read/write access. These are where your domain-specific measurement strategies live.

When you reference a collection ID in an evaluation request, EvalHub expands it: it reads the benchmark list, groups benchmarks by provider, routes each group to the appropriate backend in parallel, applies the weights, and returns a two-tier result — collection-level aggregate and per-benchmark breakdown.

The Leaderboard v2 Collection: A System Baseline Worth Understanding

Leaderboard v2 (`leaderboard-v2`) is EvalHub's implementation of the Open LLM Leaderboard v2 benchmark suite — the current standard for comparing general-purpose instruction-tuned models. It comprises six benchmarks, all executed via `lm_evaluation_harness`:

None

```
id: leaderboard-v2
category: leaderboard
description: "Leaderboard v2"
```

tags:

- leaderboard

pass_criteria:

threshold: 38.0

benchmarks:

- id: leaderboard_ifeval
provider_id: lm_evaluation_harness
metric: inst_level_strict_acc
threshold: 80.0
weight: 1
lower_is_better: false
- id: leaderboard_bbh
provider_id: lm_evaluation_harness
metric: acc_norm
threshold: 68.0
weight: 1
lower_is_better: false
- id: leaderboard_gpqa
provider_id: lm_evaluation_harness
metric: acc_norm
threshold: 40.0
weight: 1
lower_is_better: false
- id: leaderboard_mmlu_pro
provider_id: lm_evaluation_harness
metric: acc_norm
threshold: 60.0
weight: 1
lower_is_better: false
- id: leaderboard_musr
provider_id: lm_evaluation_harness

```
metric: acc_norm
threshold: 38.0
weight: 1
lower_is_better: false

- id: leaderboard_math_hard
  provider_id: lm_evaluation_harness
  metric: exact_match
  threshold: 55.0
  weight: 1
  lower_is_better: false
```

Why These Six Benchmarks

Each benchmark measures a qualitatively distinct capability:

Benchmark	What it actually measures	Key callout
IFEval	Instruction following — can the model do what it is told, in the format it is told	The largest differentiator between base and instruction-tuned models at the <80B scale
BBH	Chain-of-thought reasoning across 23 hard tasks where earlier models scored below human-level	Relatively stable across well-tuned 70B models; Phi-4 (14B) scores ~65, above many larger models
GPQA	PhD-level science reasoning in biology, physics, and chemistry	Hardest discriminator in this class — PhD experts score 65%, skilled non-experts 34%; frontier GPT-4 baselines hit only 39% at initial release

MMLU-Pro	Reasoning-centric knowledge across 12,000+ graduate-level questions in 14+ domains	Anything above 55 is competitive; Gemma-3-27B and Phi-4 land 57–62 at this tier
MuSR	Multi-step logical reasoning inside long-form narrative text (murder mysteries, etc.)	Notoriously noisy — even Qwen2.5-72B scores 11.7; above 30 is solid
Math-Lvl-5	The hardest 1,324 problems from the MATH dataset — competition math	Math-focused post-training shows clearly here; Phi-4 and Gemma-3-27B score higher per parameter than Llama-3.1-70B

Two-Tier Threshold Logic

The collection defines thresholds at two levels, each serving a different purpose.

Benchmark-level thresholds are the per-dimension gates. They encode what "strong" looks like for a competitive <80B model in each specific capability:

Benchmark	Threshold	Rationale
IFEval	80.0	Above 65 indicates consistent instruction adherence; 80 represents frontier <80B performance (Qwen2.5-72B ~86)
BBH	68.0	Top of the "strong" range for well-tuned 70B models
GPQA	40.0	Anything above 35 for a sub-80B model is strong; 40 marks the frontier tier
MMLU-Pro	60.0	Competitive bar for this tier; top models land 57–67
MuSR	38.0	Top of what the best <80B models reliably achieve
Math-Lvl-5	55.0	Strong post-training signal; above 55 is excellent for <80B

The collection-level `pass_criteria.threshold` of **38.0** is not derived from a single benchmark. It is the weighted average that a model must clear across all six to be considered passing at the

collection level. With equal weights, this is the arithmetic mean of the per-benchmark scores. The threshold of 38.0 represents a realistic "genuinely capable" bar — a model scoring above this average across all six has cleared a broader capability floor than any individual benchmark can establish.

A practical "competitive" bar that distinguishes capable from average at the <80B weight class:

IFEval $\geq$ 65 | BBH $\geq$ 55 | GPQA $\geq$ 25 | MMLU-Pro $\geq$ 50 | MuSR $\geq$ 25 | Math-Lvl-5 $\geq$ 35

The Leaderboard v2 thresholds set the *strong* bar — roughly 10–15 points higher. This is deliberate: system collections are conservative reference points, not deployment gates. Your custom collection is where you set the thresholds that actually matter for your use case.

Creating a Custom Collection

System collections, such as Leaderboard v2, are read-only. To define your own measurement strategy — different benchmarks, different weights, thresholds calibrated to your use case — you create a user-scoped collection.

The full schema for a collection create request:

```
None
{
  "name": "string (required)",
  "category": "string (required)",
  "description": "string (optional, max 1024 chars)",
  "benchmarks": [
    {
      "id": "string (required) - benchmark identifier",
      "provider_id": "string (required) - which backend runs this",
      "metric": "string - primary metric name",
      "threshold": "number - pass threshold for this benchmark",
      "weight": "number (non-negative) - contribution to aggregate, default 1",
      "lower_is_better": "boolean - score direction, default false",
      "parameters": "object (optional) - benchmark-specific config",
      "url": "string (optional) - custom provider URL"
    }
  ],
  "tags": ["string"],
  "metadata": {"key": "value"},
  "pass_criteria": {
    "threshold": "number - collection-level aggregate threshold"
  }
}
```

Worked Example: Adapting Leaderboard v2 for a Production Deployment Gate

Suppose your team is deploying a general-purpose assistant on OpenShift, and you want a deployment gate that reflects your actual requirements rather than the frontier-optimized thresholds in the system collection. Your model is expected to handle instruction-following tasks and multi-step reasoning, but you are not optimizing for PhD-level science or competition math.

You want:

- Lower stakes on GPQA and Math-Lvl-5 (not your use case's core capability)
- Higher weight on IFEval (instruction following is critical for your assistant)
- A collection-level pass bar calibrated to a "good competitive <80B" model, not frontier

None

```
name: "General Assistant Deployment Gate v1"
category: "deployment-gate"
description: "Production deployment gate for general-purpose assistants. Based on Leaderboard v2 with weights adjusted for instruction-following priority. Thresholds calibrated to competitive <80B bar."
tags:
  - assistant
  - deployment-gate
  - general-purpose
pass_criteria:
  threshold: 55.0

benchmarks:
  - id: leaderboard_ifeval
    provider_id: lm_evaluation_harness
    metric: inst_level_strict_acc
    threshold: 65.0
    weight: 2.0
    lower_is_better: false

  - id: leaderboard_bbh
    provider_id: lm_evaluation_harness
    metric: acc_norm
    threshold: 55.0
    weight: 1.5
    lower_is_better: false

  - id: leaderboard_gpqa
    provider_id: lm_evaluation_harness
    metric: acc_norm
    threshold: 25.0
```

```

weight: 0.5
lower_is_better: false

- id: leaderboard_mmlu_pro
  provider_id: lm_evaluation_harness
  metric: acc_norm
  threshold: 50.0
  weight: 1.5
  lower_is_better: false

- id: leaderboard_musr
  provider_id: lm_evaluation_harness
  metric: acc_norm
  threshold: 25.0
  weight: 1.0
  lower_is_better: false

- id: leaderboard_math_hard
  provider_id: lm_evaluation_harness
  metric: exact_match
  threshold: 35.0
  weight: 0.5
  lower_is_better: false

```

Key design decisions made explicit:

- **IFEval weight: 2.0** — instruction-following is the primary capability this deployment needs. Doubling its weight means IFEval contributes twice as much to the aggregate as lower-priority dimensions.
- **GPQA and Math-Lvl-5 weights: 0.5** — these benchmarks are still measured (regressions matter), but their contribution to the gate is halved.
- **Collection threshold: 55.0** — this is the weighted average. With IFEval weighted 2x, clearing IFEval at 65 pulls the average up significantly; failing it drags the aggregate down hard. The threshold is set to be achievable by a well-tuned competitive model that prioritizes the right capabilities.

The `pass_criteria.threshold` is intentional: it is not the average of the per-benchmark thresholds. It is a separate, independently calibrated number. A model can fail one benchmark threshold and still clear the collection-level gate if it excels in weighted-higher dimensions. A model can pass every individual benchmark threshold and still fail the collection gate if its weighted average falls short. These two signals together are more informative than either alone.

Registering the Collection via CLI

Save the YAML above as `assistant-gate-v1.yaml`, then:

None

```
evalhub collections create --spec assistant-gate-v1.yaml
```

The server validates the request, assigns an ID, timestamps it, and registers it as a user-scoped collection. The response includes the assigned `id` — use it in evaluation requests.

None

```
# Confirm the collection was registered  
evalhub collections describe <assigned-id>
```

```
# List all collections, including system and user-scoped  
evalhub collections list
```

Via the Python SDK:

None

```
from evalhub.client import SyncEvalHubClient  
  
client = SyncEvalHubClient(base_url="http://evalhub-service:8080")  
  
# List all collections  
collections = client.collections.list()  
  
# Retrieve the specific collection to verify  
collection = client.collections.get(id="<assigned-id>")
```

Running an Evaluation Against Your Collection

With the collection registered, running an evaluation is a single request that references the collection ID:

None

```
evalhub eval run \  
  --collection <assigned-id> \  
  --model-url http://vllm-service:8080/v1 \  
  --wait
```

The `--wait` flag blocks until the run completes and returns a non-zero exit code if the collection-level pass criteria is not met — making this directly usable as a CI/CD gate.

Equivalently, via the REST API:

None

```
curl -X POST http://evalhub-service:8080/api/v1/evaluations \  
  -H "Content-Type: application/json" \  
  -d '{  
    "model": { "url": "http://vllm-service:8080/v1" },  
    "collection_id": "<assigned-id>",  
    "experiment": {  
      "name": "llama-3.2-3b-assistant-gate-eval",  
      "tags": {  
        "model_family": "llama-3",  
        "environment": "staging",  
        "collection_version": "v1"  
      }  
    }  
  }'
```

EvalHub expands the collection, groups the six benchmarks (all from `lm_evaluation_harness` in this case, so a single provider call), runs them, applies the weights, and returns:

None

```
{  
  "status": "completed",  
  "collection_id": "<assigned-id>",  
  "collection_score": 61.4,  
  "pass_criteria": {  
    "threshold": 55.0,  
    "passed": true  
  },  
  "benchmark_results": [  
    { "id": "leaderboard_ifeval", "score": 71.2, "threshold": 65.0, "passed": true },  
    { "id": "leaderboard_bbh", "score": 58.3, "threshold": 55.0, "passed": true },  
    { "id": "leaderboard_gpqa", "score": 22.1, "threshold": 25.0, "passed": false },  
    { "id": "leaderboard_mmlu_pro", "score": 51.8, "threshold": 50.0, "passed": true },  
  ]  
}
```

```

    { "id": "leaderboard_musr",      "score": 29.4, "threshold": 25.0, "passed": true },
    { "id": "leaderboard_math_hard", "score": 31.2, "threshold": 35.0, "passed": false }
  ],
  "experiment": { "mlflow_run_id": "..."}
}

```

Two things are immediately visible in this result:

1. The model **cleared the collection gate** ($61.4 > 55.0$ threshold) — it is deployable by your defined criteria.
2. GPQA (22.1 vs 25.0) and Math-Lvl-5 (31.2 vs 35.0) **failed their individual thresholds** — but because these are low-weight dimensions, the failure did not sink the aggregate.

This is the distinction that matters operationally. The collection gate answers the deployment question: *Can this go to production?* The per-benchmark breakdown answers the engineering question: *Where does it still need work?* Both are present in every evaluation result. An aggregate-only result forces you to choose between them.

The full experiment record — scores, configurations, collection version, hardware tags — is written to MLflow automatically via the `tags` in the experiment config. Every run is queryable. Reproduced three months from now, you have the exact configuration that produced these numbers.

Managing Collection Evolution

Evaluation criteria are not static. Regulatory requirements change. New benchmarks emerge. Your use case evolves. The collection schema is built for this.

Update a collection via the REST API:

```

None

# Partial update - change the pass_criteria threshold only
curl -X PATCH http://evalhub-service:8080/api/v1/evaluations/collections/<id> \
  -H "Content-Type: application/json" \
  -d '[
    { "op": "replace", "path": "/pass_criteria/threshold", "value": 60.0 }
  ]'

# Full replacement - swap in a new benchmark list
curl -X PUT http://evalhub-service:8080/api/v1/evaluations/collections/<id> \
  -H "Content-Type: application/json" \
  -d '{ ... full updated collection body ... }'

```

The practical implication: when your collection changes, update the `tags` in evaluation requests to include a `collection_version` tag. MLflow then lets you query "all runs against v1 of this collection" separately from "all runs against v2." Collection evolution becomes traceable in the experiment history — not a silent config drift that makes historical scores incomparable.

What This Unlocks

Every team operating production AI workloads has evaluation criteria, whether written down or not. The criteria live in someone's head, a Slack thread, a comment in a notebook, or a one-off script that ran before the last launch. That tribal knowledge is not reusable, not auditable, and not portable across model upgrades or team changes.

An Evaluation Collection externalizes that knowledge. The thresholds are not judgment calls made the night before a launch — they are deliberate design decisions, documented in a versioned artifact, enforced consistently on every evaluation run. Changing them requires a deliberate act, not an implicit recalibration.

The Leaderboard v2 collection gives you a grounded starting point: six benchmarks whose thresholds are calibrated against the real distribution of <80B model scores, with documented rationale for each threshold choice. From there, the work is adapting weights and thresholds to your actual use case — which is the only evaluation strategy that will tell you something actionable about your specific deployment.

Start with `evalhub collections describe leaderboard-v2`. Understand what each threshold is actually measuring and why. Then build the collection that answers your deployment question — not the open-source leaderboard's question.

Start Here

- **EvalHub website:** <https://eval-hub.github.io>
- **EvalHub Server** (Collections API, OpenAPI spec): github.com/eval-hub/eval-hub
- **EvalHub SDK** (`evalhub collections` CLI, REST client):
github.com/eval-hub/eval-hub-sdk
- **OpenAPI Specification:** eval-hub.github.io/eval-hub

- **TrustyAI Operator** (Kubernetes/OpenShift deployment):
github.com/trustyai-explainability/trustyai-service-operator

Blog05: Bring Your Own Framework

EvalHub: Bring Your Own Framework — Plug Any Evaluation Tool Into the Platform

By [William Caban](#) [Rui Vieira](#) [Matteo Mortari](#)

How to onboard a custom evaluation framework into EvalHub — one class, one method, and a container image.

The Problem With "Supported Frameworks Only"

EvalHub ships with a default provider set that covers most general-purpose evaluation needs: `lm-evaluation-harness` for capability benchmarks, `Garak` for safety probes, `GuideLLM` for infrastructure profiling, `LightEval` for fast capability checks, and `MTEB` for embedding quality. For many teams, that is enough.

For many others, it is not.

Your organization may have a proprietary evaluation harness built on years of domain-specific test cases. You may use an academic framework not yet in the default set. You may have a fine-tuned judge model that scores outputs against internal rubrics. Whatever the case, if your evaluation logic is not in EvalHub's provider list, you cannot use the platform's orchestration, experiment tracking, OCI artifact persistence, or Collection-based scoring — and you are back to running evaluations outside the system.

The Bring-Your-Own-Framework (BYOF) pattern closes that gap. You implement one Python method, package it in a container image, and EvalHub treats your framework as a first-class provider: it schedules runs, tracks experiments in MLflow, persists results to OCI, and includes your scores in Collection aggregates.

The Contract: One Method

The entire BYOF adapter surface is one abstract method in one base class:

```

None

from evalhub.adapter import FrameworkAdapter, JobCallbacks, JobResults, JobSpec

class MyEvalAdapter(FrameworkAdapter):
    def run_benchmark_job(
        self,
        config: JobSpec,
        callbacks: JobCallbacks,
    ) -> JobResults:
        ...

```

Everything else — loading the job specification, authenticating with the EvalHub service, communicating with the Kubernetes sidecar, pushing OCI artifacts, writing to MLflow — is handled by the SDK. Your implementation focus is entirely inside `run_benchmark_job`.

The Three Interfaces

JobSpec — What Comes In

`JobSpec` is the complete job description EvalHub passes to your adapter at runtime. It is automatically loaded by the base class from `/meta/job.json` (Kubernetes mode) or from `./meta/job.json` (local development mode).

```

None

class JobSpec(BaseModel):
    id: str # Unique job ID from the service
    provider_id: str # Your registered provider identifier
    benchmark_id: str # Which benchmark to run
    benchmark_index: int # Index in provider's benchmark list
    model: ModelConfig # model.url + model.name (+ optional auth)
    parameters: dict # Benchmark-specific config (arbitrary)
    callback_url: str # EvalHub endpoint for status/result callbacks
    num_examples: int | None # Limit evaluation scope; None = run all
    experiment_name: str | None # MLflow experiment name
    tags: dict[str, str] | None # Labels propagated to MLflow
    exports: dict | None # OCI export coordinates
    timeout_seconds: int # Hard execution deadline

```

The `parameters` dict is where benchmark-specific configuration lives — such as few-shot count, batch size, random seed, dataset split, and anything else your framework needs. EvalHub passes it through opaquely; your adapter interprets it.

JobCallbacks — Side Effects

`JobCallbacks` is the interface for reporting progress and persisting artifacts. It has three methods:

None

```
class JobCallbacks(ABC):
    def report_status(self, update: JobStatusUpdate) -> None:
        """Send a progress update to EvalHub (or log locally in dev mode)."""

    def create_oci_artifact(self, spec: OCIArtifactSpec) -> OCIArtifactResult:
        """Push a directory of result files to an OCI registry.
        Returns the digest and full artifact reference URI."""

    def report_results(self, results: JobResults) -> None:
        """Report completion. In K8s mode, also signals the sidecar to terminate."""
```

In production, `DefaultCallbacks` implements all three against a real EvalHub instance — HTTP status callbacks, OCI registry push with the sidecar handling authentication, and MLflow integration. In local development, `DefaultCallbacks` logs to stdout and skips network calls. Same code path, different runtime behavior, no changes to your adapter.

Status updates use structured phases:

None

```
from evalhub.adapter import JobPhase, JobStatus, JobStatusUpdate

callbacks.report_status(JobStatusUpdate(
    status=JobStatus.RUNNING,
    phase=JobPhase.LOADING_DATA,
    progress=0.1,
    message=MessageInfo(message="Loading dataset", message_code="loading"),
    current_step=1,
    total_steps=4,
))
```

Phases: `INITIALIZING` → `LOADING_DATA` → `RUNNING_EVALUATION` → `POST_PROCESSING` → `PERSISTING_ARTIFACTS` → `COMPLETED`.

JobResults — What Goes Out

`JobResults` is the structured completion payload. Every field except `oci_artifact` and `mlflow_run_id` is your responsibility to populate:

None

```
class JobResults(BaseModel):
    id: str # Echo config.id
    benchmark_id: str # Echo config.benchmark_id
    benchmark_index: int # Echo config.benchmark_index
    model_name: str # Echo config.model.name
    results: list[EvaluationResult] # One entry per metric
    overall_score: float | None # Aggregate score (0-100)
    num_examples_evaluated: int # Count of samples processed
    duration_seconds: float # Wall-clock execution time
    completed_at: datetime # UTC completion timestamp
    evaluation_metadata: dict[str, Any] # Framework version, params, etc.
    oci_artifact: OCIArtifactResult | None # Filled by callbacks.create_oci_artifact
    mlflow_run_id: str | None # Filled by the SDK automatically
```

EvaluationResult is the per-metric type:

None

```
class EvaluationResult(BaseModel):
    metric_name: str # e.g., "accuracy", "f1", "bleu"
    metric_value: float | int | str | bool # The measured value
    metric_type: str = "float" # Type classification
    confidence_interval: tuple[float, float] | None = None
    num_samples: int | None = None
    metadata: dict[str, Any] = {}
```

A Complete Minimal Adapter

This is the full pattern for a working adapter — replace the `_run_my_framework` call with your actual evaluation logic:

None

```
import time
from datetime import UTC, datetime
from pathlib import Path

from evalhub.adapter import (
    FrameworkAdapter,
    JobCallbacks,
    JobPhase,
    JobResults,
    JobSpec,
    JobStatusUpdate,
    MessageInfo,
    OCIArtifactSpec,
    OCICoordinates
)
from evalhub.adapter.callbacks import DefaultCallbacks
from evalhub.models import EvaluationResult, JobStatus

class MyEvalAdapter(FrameworkAdapter):
```

```

def run_benchmark_job(self, config: JobSpec, callbacks: JobCallbacks) -> JobResults:
    start = time.monotonic()

    # --- Phase 1: Initialize ---
    callbacks.report_status(JobStatusUpdate(
        status=JobStatus.RUNNING,
        phase=JobPhase.INITIALIZING,
        progress=0.0,
        message=MessageInfo(message="Initializing", message_code="init"),
    ))

    # --- Phase 2: Load data ---
    callbacks.report_status(JobStatusUpdate(
        status=JobStatus.RUNNING,
        phase=JobPhase.LOADING_DATA,
        progress=0.2,
        message=MessageInfo(message="Loading dataset", message_code="loading"),
    ))
    # config.parameters holds your benchmark-specific config
    num_examples = config.num_examples or 500
    dataset = load_my_dataset(config.benchmark_id, num_examples)

    # --- Phase 3: Run evaluation ---
    callbacks.report_status(JobStatusUpdate(
        status=JobStatus.RUNNING,
        phase=JobPhase.RUNNING_EVALUATION,
        progress=0.4,
        message=MessageInfo(message="Running evaluation", message_code="running"),
    ))
    raw_results = _run_my_framework(
        model_url=config.model.url,
        model_name=config.model.name,
        dataset=dataset,
        params=config.parameters,
    )

    # --- Phase 4: Post-process ---
    callbacks.report_status(JobStatusUpdate(
        status=JobStatus.RUNNING,
        phase=JobPhase.POST_PROCESSING,
        progress=0.8,
        message=MessageInfo(message="Computing metrics", message_code="postproc"),
    ))
    metrics = [
        EvaluationResult(metric_name="accuracy", metric_value=raw_results["acc"]),
        EvaluationResult(metric_name="f1", metric_value=raw_results["f1"]),
    ]
    overall = raw_results["acc"] * 100

    # --- Phase 5: Persist artifacts ---
    callbacks.report_status(JobStatusUpdate(
        status=JobStatus.RUNNING,
        phase=JobPhase.PERSISTING_ARTIFACTS,
        progress=0.9,
        message=MessageInfo(message="Persisting artifacts", message_code="oci"),
    ))
    artifacts_dir = self.local_jobs_base_path or Path("/tmp/results")
    artifacts_dir.mkdir(parents=True, exist_ok=True)
    (artifacts_dir / "results.json").write_text(str(raw_results))

    oci_result = callbacks.create_oci_artifact(OCIArtifactSpec(
        files_path=artifacts_dir,
        coordinates=OCICoordinates(
            oci_host="guay.io",
            oci_repository="my-org/my-framework-results",
        ),
    ),
    )

```

```

    return JobResults(
        id=config.id,
        benchmark_id=config.benchmark_id,
        benchmark_index=config.benchmark_index,
        model_name=config.model.name,
        results=metrics,
        overall_score=overall,
        num_examples_evaluated=len(dataset),
        duration_seconds=time.monotonic() - start,
        completed_at=datetime.now(UTC),
        evaluation_metadata={"framework": "my-framework", "version": "1.0"},
        oci_artifact=oci_result,
    )

def main():
    adapter = MyEvalAdapter()
    callbacks = DefaultCallbacks.from_adapter(adapter)
    results = adapter.run_benchmark_job(adapter.job_spec, callbacks)
    callbacks.report_results(results)

if __name__ == "__main__":
    main()

```

`self.local_jobs_base_path` returns a run-scoped directory in local mode (preventing result collisions across concurrent local runs) and `None` in Kubernetes mode — in which case your adapter should write to a stable path like `/tmp/results` that the sidecar can access.

Packaging as a Container

EvalHub executes BYOF adapters as Kubernetes Jobs. The container pattern from the community adapters in `eval-hub-contrib` is:

```

None
FROM registry.access.redhat.com/ubi9/python-311:latest

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .

# Non-root user required for OpenShift

```

```
RUN mkdir -p /tmp/.cache && chmod -R 777 /tmp && chmod -R g+w /app  
/opt/app-root  
USER 1001  
  
ENV PYTHONUNBUFFERED=1  
ENV EVALHUB_MODE=k8s  
ENV HOME=/tmp  
  
ENTRYPOINT ["python", "main.py"]
```

Two environment variables matter:

- `EVALHUB_MODE=k8s` — tells the SDK to load `JobSpec` from `/meta/job.json` (mounted by EvalHub as a `ConfigMap`) and use the sidecar for callbacks
- `EVALHUB_MODE=local` (default) — loads from `./meta/job.json` and logs callbacks locally; used for development and testing

Push the image to a registry accessible from your cluster:

```
None  
podman build -t quay.io/my-org/my-eval-framework:latest .  
podman push quay.io/my-org/my-eval-framework:latest
```

Community adapters follow the naming convention `quay.io/evalhub/community-{framework}:latest` — your custom adapters can use any registry path your cluster can pull from.

The Kubernetes Execution Model

Understanding what EvalHub does when scheduling your adapter helps avoid surprises.

When EvalHub dispatches a job to your provider, it creates a Kubernetes Job with a two-container pod:

Pod

Adapter Container

```
your main.py  
reads job.json  
runs benchmark  
calls callbacks  
writes term file
```

Sidecar Container

```
HTTP callback server  
OCI registry auth  
artifact push  
watches for /shared/terminated
```

/meta/job.json ← **ConfigMap** mount (read-only)

/shared/ ← **EmptyDir** volume (read/write, shared)

The `JobSpec` arrives via `/meta/job.json`. Status callbacks from `callbacks.report_status()` POST to the sidecar's local HTTP server, which forwards them to EvalHub. Artifact pushes from `callbacks.create_oci_artifact()` delegates registry authentication to the sidecar. `callbacks.report_results()` writes the termination file to `/shared/terminated`, signaling the sidecar to shut down. You do not need to manage any of this directly — `DefaultCallbacks` handles it.

In local development, none of this runs. `DefaultCallbacks` detects `EVALHUB_MODE=local`, logs all callbacks to stdout, and skips network calls. To test locally before building a container:

None

Create the job spec in the expected location

```
mkdir -p meta
```

```
cat > meta/job.json << 'EOF'
```

```
{
```

```
  "id": "local-test-001",
```

```
  "provider_id": "my-framework",
```

```
  "benchmark_id": "my-benchmark",
```

```
  "benchmark_index": 0,
```

```
  "model": {
```

```
    "url": "http://localhost:8000/v1",
```

```

    "name": "my-model"
  },
  "parameters": {"batch_size": 4},
  "callback_url": "http://localhost:8080",
  "timeout_seconds": 3600
}
EOF

```

```
EVALHUB_MODE=local python main.py
```

Registering Your Provider

Once your container image is built and pushed, register the provider with EvalHub via a ConfigMap entry in the server's configuration. A provider registration declares the image to use, the benchmarks it supports, and the parameters each benchmark accepts.

After registration, your provider appears in `evalhub providers list` alongside the built-in providers, and benchmark IDs from your adapter can be included in Evaluation Collections exactly like any built-in benchmark:

```

None
# Collection referencing your custom provider
name: "My Domain Eval Suite v1"
category: "domain-specific"
pass_criteria:
  threshold: 70.0
benchmarks:
  - id: my-benchmark
    provider_id: my-framework      # Your registered provider ID
    metric: accuracy
    threshold: 75.0
    weight: 2.0
    lower_is_better: false
  - id: leaderboard_ifeval        # Mix with built-in benchmarks
    provider_id: lm_evaluation_harness
    metric: inst_level_strict_acc
    threshold: 65.0
    weight: 1.0
    lower_is_better: false

```

Each benchmark entry specifies which metric to use for scoring, the pass/fail threshold, and a weight for the collection's aggregate score. `lower_is_better: false` means higher values are better (the default).

What You Get for Free

Implementing `run_benchmark_job` and packaging the container is the entirety of the BYOF integration work. In return, your adapter inherits everything the platform provides to built-in providers:

- **MLflow experiment tracking** — every run is automatically recorded with the job configuration, tags, model info, and your returned metrics
- **OCI artifact persistence** — results pushed to a registry with SHA256-derived tags; the artifact reference is queryable alongside MLflow data
- **Collection scoring** — your benchmark's score participates in weighted Collection aggregation; a single `pass_criteria.threshold` gates deployment
- **Kubernetes orchestration** — Kueue-managed resource quotas, pod scheduling, status tracking via Custom Resources
- **SDK client access** — `evalhub eval run`, the Python client, and the MCP server all work against your provider without changes
- **Local-to-cluster continuity** — the same `main.py` runs locally with `EVALHUB_MODE=local` and on OpenShift with `EVALHUB_MODE=k8s`

The community adapters in `eval-hub-contrib` — `LightEval`, `MTEB`, `GuideLLM` — are working reference implementations for every pattern covered here.

Start Here

- **EvalHub website:** <https://eval-hub.github.io>
- **EvalHub SDK** (FrameworkAdapter, JobSpec, DefaultCallbacks):
github.com/eval-hub/eval-hub-sdk
- **eval-hub-contrib** (`LightEval`, `MTEB`, `GuideLLM` reference adapters):
github.com/eval-hub/eval-hub-contrib
- **EvalHub Server** (provider registration, Collections API): github.com/eval-hub/eval-hub

Blog06: EvalHub from the CLI

EvalHub from the CLI: Integrate AI Evaluations Into Your CI/CD Pipeline

By [William Caban](#) [Rui Vieira](#) [Matteo Mortari](#)

The complete practical guide to running EvalHub evaluations from the terminal and wiring them into pipeline gates.

Why the CLI Matters for DevOps

EvalHub's API server and Kubernetes operator handle orchestration. The Python SDK handles notebook and application integration. But for CI/CD pipelines, the CLI is the right surface: it installs in seconds, reads config from environment variables, returns machine-parseable output, and exits non-zero on failure.

This post covers the full CLI workflow — from first-time setup through a production pipeline gate — without detours into platform architecture or evaluation methodology. Those are covered in the rest of the series. This is the operational reference.

Installation

The CLI ships as an optional dependency group of the EvalHub SDK:

```
None  
pip install "eval-hub-sdk[cli]"
```

Verify the install and confirm connectivity to your EvalHub instance:

```
None  
evalhub version  
evalhub health
```

`evalhub health` exits 0 if the service is reachable, 1 if it is not. Use it as the first step in any pipeline that depends on EvalHub.

Configuration

Config File

The CLI stores named profiles in `~/.config/evalhub/config.yaml`:

None

```
active_profile: default

profiles:
  default:
    base_url: http://evalhub-service:8080
    token: ""
    tenant: ""

  staging:
    base_url: https://evalhub.staging.example.com
    token: "staging-token"
    tenant: "team-platform"
    timeout: 60

  prod:
    base_url: https://evalhub.prod.example.com
    token: "prod-token"
    tenant: "team-platform"
    insecure: false
    timeout: 120
```

Required per profile: `base_url`, `token`, `tenant`. Set values with:

None

```
evalhub config set base_url http://evalhub-service:8080
evalhub config set token <your-token>
evalhub config set tenant ""
```

```
# Add a second profile
EVALHUB_PROFILE=prod evalhub config set base_url https://evalhub.prod.example.com
evalhub config use prod # switch active profile
```

Environment Variables

In CI/CD, prefer environment variables over config files — they integrate directly with secret management:

Variable	Purpose	Overrides
EVALHUB_BASE_URL	Server URL	profile <code>base_url</code>
EVALHUB_TOKEN	Auth token	profile <code>token</code>
EVALHUB_PROFILE	Active profile name	config <code>active_profile</code>
EVALHUB_VERBOSE	Enable debug logging	—
EVALHUB_CONFIG	Custom config file path	default path

Priority order: CLI flag → environment variable → config file profile → default.

In any pipeline, setting `EVALHUB_BASE_URL` and `EVALHUB_TOKEN` from secrets is sufficient. No config file needed.

Running Evaluations

The eval.yaml Config File

For repeatable pipeline runs, define the evaluation in a YAML file committed to your repository:

```
None
# eval.yaml
name: "llama-3.2-staging-gate"
description: "Pre-merge evaluation gate for staging deployments"

model:
  url: "http://vllm-service:8000/v1"
  name: "meta-llama/Llama-3.2-3B-Instruct"

collection:
  id: "general-assistant-gate-v1"      # reference a registered
collection

experiment:
  name: "llama-3.2-staging-eval"
  tags: # MLflow tags
    - key: "environment"
      value: "staging"
    - key: "trigger"
      value: "pre-merge"
```

To run against individual benchmarks instead of a collection:

```
None
name: "targeted-benchmark-run"
model:
  url: "http://vllm-service:8000/v1"
  name: "meta-llama/Llama-3.2-3B-Instruct"

benchmarks:
  - id: "leaderboard_ifeval"
    provider_id: "lm_evaluation_harness"
    parameters:
      metrics:
```

```

        - inst_level_strict_acc
- id: "leaderboard_bbh"
  provider_id: "lm_evaluation_harness"

exports:
  oci:
    coordinates:
      oci_host: quay.io
      oci_repository: my-org/eval-results

```

`benchmarks` and `collection` are mutually exclusive — use one or the other.

Submit and Wait

None

```

# Submit and return immediately (async)
evalhub eval run --config eval.yaml

```

```

# Submit and block until completion (use this in pipelines)
evalhub eval run --config eval.yaml --wait --timeout 3600

```

`--wait` polls the job status until it reaches a terminal state (completed or failed). `--timeout` sets the maximum wait in seconds. The command exits 1 if the job fails — making it a direct pipeline gate.

To capture the job ID for subsequent commands:

None

```

JOB_ID=$(evalhub eval run --config eval.yaml --format json | jq -r
'.[0].id')
echo "Job submitted: $JOB_ID"

```

Checking Status

None

List all jobs

```
evalhub eval status
```

Filter by state

```
evalhub eval status --status running
```

```
evalhub eval status --status completed
```

Filter by recency (client-side)

```
evalhub eval status --since 24h
```

```
evalhub eval status --provider lm_evaluation_harness --since 7d
```

Single job detail

```
evalhub eval status $JOB_ID
```

Block until terminal state (for async submissions)

```
evalhub eval status $JOB_ID --watch --poll-interval 10
```

Retrieving Results

None

Human-readable table (default)

```
evalhub eval results $JOB_ID
```

Machine-readable JSON

```
evalhub eval results $JOB_ID --format json
```

CSV for export

```
evalhub eval results $JOB_ID --format csv > results.csv
```

Table output:

None

benchmark	provider	metric	value
leaderboard_ifeval	lm_evaluation_harness	inst_level_strict_acc	0.712
leaderboard_bbh	lm_evaluation_harness	acc_norm	0.581

MLflow experiment: <https://mlflow.example.com/experiments/42>

Managing Collections from the CLI

None

```
# List all available collections (system + user)
evalhub collections list
evalhub collections list --tag leaderboard --format json

# Inspect a collection
evalhub collections describe leaderboard-v2

# Create a user collection from a spec file
evalhub collections create --file my-collection.yaml

# Run a collection directly (shorthand for eval run with collection
reference)
evalhub collections run general-assistant-gate-v1 \
  --model-url http://vllm-service:8000/v1 \
  --model-name meta-llama/Llama-3.2-3B-Instruct \
  --wait \
  --timeout 3600
```

Cancelling Jobs

None

```
evalhub eval cancel $JOB_ID # cancel with confirmation
prompt
evalhub eval cancel $JOB_ID --hard-delete # permanent deletion, no
recovery
```

CI/CD Pipeline Integration

GitHub Actions

None

```
# .github/workflows/model-eval.yaml
```

```
name: Model Evaluation Gate
```

```
on:
```

```
  pull_request:
```

```
    paths:
```

- 'model/**'
- 'prompts/**'
- 'eval.yaml'

```
jobs:
```

```
  evaluate:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

- uses: actions/checkout@v4
- name: Install EvalHub CLI
 run: pip install "eval-hub-sdk[cli]"
- name: Verify service connectivity
 env:
 EVALHUB_BASE_URL: \${ secrets.EVALHUB_BASE_URL }
 EVALHUB_TOKEN: \${ secrets.EVALHUB_TOKEN }
 run: evalhub health
- name: Run evaluation gate
 env:
 EVALHUB_BASE_URL: \${ secrets.EVALHUB_BASE_URL }
 EVALHUB_TOKEN: \${ secrets.EVALHUB_TOKEN }
 run: |
 evalhub eval run \
 --config eval.yaml \
 --wait \
 --timeout 3600
- name: Export results
 if: always()
 env:
 EVALHUB_BASE_URL: \${ secrets.EVALHUB_BASE_URL }
 EVALHUB_TOKEN: \${ secrets.EVALHUB_TOKEN }
 run: |
 JOB_ID=\$(evalhub eval status --status completed --since 1h --format json \
\
 | jq -r '.[0].id')
 evalhub eval results "\$JOB_ID" --format json > eval-results.json


```

    evalhub eval results "$JOB_ID" --format csv > eval-results.csv

- name: Upload results artifact
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: eval-results-${{ github.sha }}
    path: |
      eval-results.json
      eval-results.csv

```

`evalhub eval run --wait` exits 1 if the collection gate fails, which fails the GitHub Actions step and blocks the PR merge. No post-processing script needed for the gate itself.

GitLab CI

```

None
# .gitlab-ci.yml
model-evaluation:
  stage: test
  image: python:3.11-slim
  variables:
    EVALHUB_BASE_URL: $EVALHUB_BASE_URL  # from GitLab CI/CD variables
    EVALHUB_TOKEN: $EVALHUB_TOKEN
  before_script:
    - pip install "eval-hub-sdk[cli]"
    - evalhub health
  script:
    - evalhub eval run --config eval.yaml --wait --timeout 3600
  after_script:
    - |
      JOB_ID=$(evalhub eval status --status completed --since 1h --format json \
        | jq -r '.[0].id')
      evalhub eval results "$JOB_ID" --format csv > eval-results.csv
  artifacts:
    when: always
    paths:
      - eval-results.csv
    expire_in: 30 days
  rules:
    - changes:
        - model/**

```

- prompts/**
- eval.yaml

Reusable Shell Script

For pipelines that need more control — capturing the job ID, branching on per-benchmark results, posting summaries — use a script:

None

```
#!/usr/bin/env bash
# eval-gate.sh - Submit an EvalHub evaluation and gate on the result
set -euo pipefail

EVAL_CONFIG="${1:-eval.yaml}"
TIMEOUT="${2:-3600}"
RESULTS_FILE="eval-results-$(date +%Y%m%d-%H%M%S).json"

# Requires: EVALHUB_BASE_URL and EVALHUB_TOKEN in environment

echo "==> Checking EvalHub connectivity"
evalhub health

echo "==> Submitting evaluation: $EVAL_CONFIG"
JOB_ID=$(evalhub eval run --config "$EVAL_CONFIG" --format json | jq -r
'.[0].id')
echo "    Job ID: $JOB_ID"

echo "==> Waiting for completion (timeout: ${TIMEOUT}s)"
evalhub eval status "$JOB_ID" --watch --poll-interval 15

echo "==> Fetching results"
evalhub eval results "$JOB_ID" --format json > "$RESULTS_FILE"
evalhub eval results "$JOB_ID" # human-readable summary to stdout

echo "==> Results saved to $RESULTS_FILE"
```

`set -euo pipefail` combined with `evalhub eval status --watch` propagates the exit code correctly: if the job fails, `--watch` exits 1, `set -e` halts the script, and the pipeline step fails.

CLI Reference at a Glance

Command	Common Flags	Purpose
<code>evalhub health</code>	<code>—</code>	Verify service connectivity
<code>evalhub eval run</code>	<code>--config</code> , <code>--wait</code> , <code>--timeout</code> , <code>--format</code>	Submit evaluation job
<code>evalhub eval status</code>	<code>--watch</code> , <code>--status</code> , <code>--since</code> , <code>--format</code>	List or monitor jobs
<code>evalhub eval results</code>	<code>--format</code>	Retrieve completed job results
<code>evalhub eval cancel</code>	<code>--hard-delete</code>	Cancel or delete a job
<code>evalhub collections list</code>	<code>--tag</code> , <code>--format</code>	List available collections
<code>evalhub collections describe</code>	<code>--format</code>	Inspect a collection
<code>evalhub collections create</code>	<code>--file</code>	Register a new collection
<code>evalhub collections run</code>	<code>--model-url</code> , <code>--model-name</code> , <code>--wait</code>	Run a collection directly

<code>evalhub</code> <code>list</code>	<code>providers</code>	<code>--format</code>	List providers	registered
<code>evalhub</code> <code>describe</code>	<code>providers</code>	<code>--format</code>	Inspect providers	registered
<code>evalhub config set</code>		KEY VALUE	Set a config value	
<code>evalhub config use</code>		PROFILE	Switch active profile	

All data-returning commands support `--format table` (default), `json`, `yaml`, and `csv`. JSON output is stable and safe to pipe into `jq`.

Practical Notes

Use `--wait` for gates, `--watch` for monitoring. `evalhub eval run --wait` submits and blocks in a single command — simplest for pipelines. `evalhub eval status $JOB_ID --watch` monitors a previously submitted async job — useful when the submit step and the wait step run in different pipeline stages.

Never store tokens in the config file in CI. Set `EVALHUB_TOKEN` from your secret store. The environment variable automatically takes priority over the config file.

Scope `eval.yaml` to your pipeline trigger. A collection reference in `eval.yaml` means threshold changes are picked up automatically when the collection is updated. Benchmark lists require a file change to update.

Results are always in MLflow. Every `evalhub eval run` call automatically writes a full experiment record. `evalhub eval results` is a convenience command; the authoritative long-term store is MLflow.

Start Here

- **EvalHub website:** <https://eval-hub.github.io>

- **EvalHub SDK** (`evalhub` CLI): github.com/eval-hub/eval-hub-sdk
- **EvalHub Server** (Collections API, provider registration): github.com/eval-hub/eval-hub
- **TrustyAI Operator** (Kubernetes/OpenShift deployment):
github.com/trustyai-explainability/trustyai-service-operator

Blog07: Eval Results as Immutable OCI Artifacts

EvalHub: Evaluation Results as Immutable OCI Artifacts

By [William Caban](#) [Matteo Mortari](#)

How EvalHub pushes evaluation results to OCI registries — deterministic tags, tamper-evident digests, and transparent authentication — so the evidence that justified a deployment is never just a log entry.

The Governance Gap That MLflow Alone Cannot Close

[Because "Looks Good to Me"](#) identified the reproducibility crisis as a structural failure in enterprise AI evaluation: benchmark scores without the environment metadata that produced them are claims, not evidence. [EvalHub: Stop Guessing. Start Measuring.](#) showed how EvalHub's automatic MLflow integration closes that gap for experiment tracking — every evaluation run is recorded with its configuration, model version, collection version, and hardware tags.

MLflow solves the **queryability** problem. It does not solve the **immutability** problem.

An MLflow experiment record is mutable. Entries can be deleted, overwritten, or lost if the tracking server is rebuilt. For internal iteration, that is fine. For regulated workloads — EU AI Act, FedRAMP High, SOC 2 — the evidence that a model met its evaluation criteria before deployment needs to be tamper-evident, content-addressed, and independently verifiable. A database row does not satisfy that requirement. A signed OCI artifact in a content-addressable registry does.

EvalHub's OCI persistence layer pushes evaluation result artifacts to any OCI-compliant registry at the end of every evaluation run. The artifact reference — a digest of the form `sha256:...` — is immutable: if the contents change, the digest changes. The digest is stored in the `JobResults` and written to MLflow alongside the experiment record. The full provenance chain is: evaluation run → MLflow experiment (queryable) → OCI artifact (immutable).

How OCI Persistence Works

OCI persistence is opt-in, configured per evaluation job via the `exports` block. When configured, the adapter calls `callbacks.create_oci_artifact()` after the evaluation completes, which delegates to the `OCIArtifactPersister` internally. The persister:

1. Takes the results directory (any files your adapter wrote — JSON metrics, raw outputs, logs)
2. Creates an OCI artifact layout using `olot`
3. Authenticates to the registry (via Docker config in local mode, via Kubernetes sidecar in cluster mode)
4. Pushes the artifact using `oras`
5. Retrieves the `Docker-Content-Digest` response header
6. Returns an `OCIArtifactResult` with the digest and full artifact reference URI

The result is a content-addressed, immutable artifact at a stable reference like:

None

```
quay.io/my-org/eval-results:evalhub-a3f7c1b2@sha256:9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a3
```

That reference is stored in `JobResults.oci_artifact`, reported back to the EvalHub server, and written as MLflow artifact metadata. Pulling that reference six months from now returns exactly the results that were produced by that evaluation run — or fails with a digest mismatch if anything has been altered.

Configuring OCI Export

In `eval.yaml`

Add an `exports` block to your evaluation config file:

None

```
name: "llama-3.2-staging-gate"
model:
  url: "http://vllm-service:8000/v1"
  name: "meta-llama/Llama-3.2-3B-Instruct"
collection:
```



```

id: "general-assistant-gate-v1"

exports:
  oci:
    coordinates:
      oci_host: "quay.io"
      oci_repository: "my-org/eval-results"
      oci_tag: "llama-3.2-staging-2026-04-01" # optional; auto-generated if omitted
    annotations:
      environment: "staging"
      model-family: "llama-3"
      collection-version: "v1"
  k8s:
    connection: "registry-credentials" # name of K8s Secret (type: dockerconfigjson)

```

`k8s.connection` is required in Kubernetes mode — it names the `kubernetes.io/dockerconfigjson` Secret that holds registry credentials. In local mode, omit the `k8s` block and the persister reads from `~/.docker/config.json`.

OCI Coordinates Fields

Field	Required	Description
<code>oci_host</code>	Yes	Registry hostname (e.g., <code>quay.io</code> , <code>registry.example.com</code>)
<code>oci_repository</code>	Yes	Repository path (e.g., <code>my-org/eval-results</code>)
<code>oci_tag</code>	No	Custom tag; deterministic SHA256 tag generated if omitted
<code>oci_subject</code>	No	Optional subject identifier within the same registry/repo
<code>annotations</code>	No	Custom key-value metadata merged into OCI annotations

In the REST API

The same `exports` structure is accepted in `POST /api/v1/evaluations/jobs` directly:

None

```
curl -X POST http://evalhub-service:8080/api/v1/evaluations/jobs \
-H "Content-Type: application/json" \
-d '{
  "name": "llama-3.2-staging-gate",
  "model": {
    "url": "http://vllm-service:8000/v1",
    "name": "meta-llama/Llama-3.2-3B-Instruct"
  },
  "collection_id": "general-assistant-gate-v1",
  "exports": {
    "oci": {
      "coordinates": {
        "oci_host": "quay.io",
        "oci_repository": "my-org/eval-results",
        "annotations": {
          "environment": "staging",
          "model-family": "llama-3"
        }
      },
    },
    "k8s": {
      "connection": "registry-credentials"
    }
  },
  "experiment": {
    "name": "llama-3.2-staging-eval",
    "tags": { "environment": "staging" }
  }
}'
```

Deterministic Tags

If `oci_tag` is omitted, the persister generates a deterministic tag from the evaluation context:

None

```
SHA256(job_id + provider_id + benchmark_id + benchmark_index)
```

The result is a hex string conforming to OCI tag specifications (alphanumeric, underscores, periods, hyphens; max 128 characters), prefixed with `evalhub-`:

None

evalhub-a3f7c1b2d4e6f8a0b2c4d6e8f0a2b4c6d8e0f2a4b6c8d0e2f4a6b8c0d2e4f6

Deterministic tags mean the same job configuration always produces the same tag — making it possible to find the artifact for a specific evaluation run without storing the reference externally. The digest still changes if the results change, providing tamper evidence; the tag provides human-navigable indexing.

Annotations

Every pushed artifact carries a set of standard OCI annotations merged with any user-provided ones. Default annotations:

Annotation	Value
<code>org.opencontainers.image.created</code>	ISO 8601 timestamp of the push
<code>io.github.eval-hub.job</code>	Job ID
<code>io.github.eval-hub.benchmark</code>	Benchmark ID
<code>io.github.eval-hub.provider</code>	Provider ID (if present)

User-provided annotations in `coordinates.annotations` take precedence over defaults. This is where you attach deployment-relevant metadata — environment, model family, collection version, compliance tags — that makes the artifact queryable in your registry's search interface.

Authentication: K8s Sidecar vs Local Docker Config

Kubernetes Mode

In Kubernetes mode (`EVALHUB_MODE=k8s`), the EvalHub-managed pod runs two containers: the adapter and a sidecar. The sidecar acts as an authenticated proxy for registry operations:

None

Adapter container

- `OCIArtifactPersister.persist()`
- routes push through sidecar proxy (`localhost:8080`)
- sidecar performs bearer token challenge with actual registry
- sidecar uses credentials from the K8s Secret named in `k8s.connection`
- push completes; digest returned to adapter
- artifact reference uses original registry host (not the proxy address)

The credentials Secret must be of type `kubernetes.io/dockerconfigjson`:

None

```
apiVersion: v1
kind: Secret
metadata:
  name: registry-credentials
  namespace: evalhub
type: kubernetes.io/dockerconfigjson
data:
  .dockerconfigjson: |
    {
      "auths": {
        "quay.io": {
          "auth": "<base64(username:password)>"
        }
      }
    }
```

The adapter code never handles registry credentials directly. Authentication is fully delegated to the sidecar, which reads fresh Kubernetes ServiceAccount tokens per request and handles token rotation automatically.

Local Development Mode

In local mode (`EVALHUB_MODE=local`, the default), the persister reads Docker credentials from `~/.docker/config.json` (or the path in `DOCKER_CONFIG`). Log in to your registry before running:

```
None
docker login quay.io
EVALHUB_MODE=local python main.py
```

No other configuration change is needed. The same adapter code runs in both modes.

Retrieving Artifact References

The OCI artifact reference and digest are returned in the evaluation result:

```
None
# Full result including OCI artifact reference
evalhub eval results $JOB_ID --format json
```

```
None
[
  {
    "id": "eval-abc123",
    "benchmark_id": "leaderboard_ifeval",
    "model_name": "meta-llama/Llama-3.2-3B-Instruct",
    "results": [
      { "metric_name": "inst_level_strict_acc", "metric_value": 0.712 }
    ],
    "overall_score": 71.2,
    "oci_artifact": {
      "digest": "sha256:9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a3",
      "oci_ref": "quay.io/my-org/eval-results:evalhub-a3f7c1b2@sha256:9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a3"
    },
    "mlflow_run_id": "3f7a2c1b4d6e8f0a"
  }
]
```

```
]
```

The `oci_ref` is the complete, pullable reference. The `digest` is the tamper-evident fingerprint. Both are written to the MLflow experiment record alongside the metrics, so querying "all evaluation runs that produced an artifact for model X in collection version Y" is a standard MLflow query — and the result includes a pullable artifact reference for each.

Pulling the Artifact

None

```
# Pull and inspect using oras
oras pull quay.io/my-org/eval-results:evalhub-a3f7c1b2@sha256:9f86d08... \
  --output ./retrieved-results

# Verify digest before using
echo "9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a3 retrieved-results/results.json" \
  | sha256sum --check
```

The digest check is the verification step: if the file's hash matches the digest in the artifact reference, the results are exactly what was produced by the evaluation run. If they do not match, the artifact has been tampered with or corrupted.

End-to-End: Evaluation Run to Auditable Artifact

None

```
# 1. Configure OCI export in eval.yaml
#   (add exports.oci block with coordinates and k8s.connection)

# 2. Run evaluation - OCI push happens automatically at job completion
evalhub eval run --config eval.yaml --wait --timeout 3600

# 3. Retrieve the artifact reference from results
evalhub eval results $JOB_ID --format json | \
  jq -r '.[0].oci_artifact.oci_ref'
# → quay.io/my-org/eval-results:evalhub-a3f7c1b2@sha256:9f86d081...

# 4. Store the reference in your deployment record / SBOM / audit log
ARTIFACT_REF=$(evalhub eval results $JOB_ID --format json | \
  jq -r '.[0].oci_artifact.oci_ref')
```

```
# 5. At any future point, pull and verify
oras pull "$ARTIFACT_REF" --output ./audit-evidence
```

In a CI/CD pipeline, step 4 writes the artifact reference into the deployment manifest or signed SBOM. A compliance audit then follows the reference from the deployed model version back to the immutable evaluation artifact — no reconstruction needed, no log parsing, no reliance on a queryable database that may have changed.

Integrating OCI Export Into a Pipeline Gate

```
None
# eval.yaml with OCI export
name: "llama-3.2-pr-gate"
model:
  url: "http://vllm-service:8000/v1"
  name: "meta-llama/Llama-3.2-3B-Instruct"
collection:
  id: "general-assistant-gate-v1"
exports:
  oci:
    coordinates:
      oci_host: "quay.io"
      oci_repository: "my-org/eval-results"
    annotations:
      git-commit: "${GIT_COMMIT}"
      pr-number: "${PR_NUMBER}"
  k8s:
    connection: "registry-credentials"
```

None

```
# .github/workflows/model-eval.yaml
- name: Run evaluation gate with OCI export
  env:
    EVALHUB_BASE_URL: ${ secrets.EVALHUB_BASE_URL }
    EVALHUB_TOKEN: ${ secrets.EVALHUB_TOKEN }
  run: |
    # Substitute commit and PR into annotations
    sed -i "s/\${GIT_COMMIT}/${ github.sha }/g" eval.yaml
    sed -i "s/\${PR_NUMBER}/${ github.event.number }/g" eval.yaml

    evalhub eval run --config eval.yaml --wait --timeout 3600

- name: Capture artifact reference
  if: success()
  env:
    EVALHUB_BASE_URL: ${ secrets.EVALHUB_BASE_URL }
    EVALHUB_TOKEN: ${ secrets.EVALHUB_TOKEN }
  run: |
    JOB_ID=$(evalhub eval status --status completed --since 1h --format json \
    | jq -r '.[0].id')
    ARTIFACT_REF=$(evalhub eval results "$JOB_ID" --format json \
    | jq -r '.[0].oci_artifact.oci_ref')
    echo "EVAL_ARTIFACT_REF=$ARTIFACT_REF" >> $GITHUB_ENV
    echo "Evaluation artifact: $ARTIFACT_REF"
```

The `git-commit` annotation in the OCI artifact links the immutable evaluation evidence directly to the source commit. Anyone auditing that deployment can pull the artifact by reference and verify the evaluation results that preceded it.

What This Gives You

Concern	Without OCI persistence		With OCI persistence
Evidence durability	MLflow (mutable)	database	Content-addressed OCI artifact (immutable)
Tamper detection	None	SHA256 digest mismatch on pull	

Evidence retrieval	Query MLflow (requires live server)	<code>oras pull <reference></code> (requires only registry access)
Audit trail	Log entries and dashboard screenshots	Pullable, verifiable artifact with structured annotations
Compliance reporting	Manual reconstruction	Reference in deployment manifest → <code>oras pull</code> → structured result files

The MLflow integration and OCI persistence are complementary, not alternatives. MLflow gives you queryability across runs — trend analysis, regression detection, experiment comparison. OCI gives you durability and verifiability for each result. Use both.

Start Here

- **EvalHub website:** <https://eval-hub.github.io>
- **EvalHub SDK** (OCI persistence, `OCIArtifactPersister`, `DefaultCallbacks`): github.com/eval-hub/eval-hub-sdk
- **eval-hub-contrib** (reference adapters showing OCI persistence in context): github.com/eval-hub/eval-hub-contrib
- **EvalHub Server** (jobs API, exports field): github.com/eval-hub/eval-hub
- **ORAS** (OCI artifact push/pull tool): oras.land